Rapport Automatique & Réseaux

Auteurs : *MUSUMECI Lucas - COUDERC Juliette*

Date : *22 avril 2026*

Polytech Montpellier

Git:
gitlab.polytech.umontpellier.fr/lucas.musumeci/controle_distant_bras_coppeliasim

Table des matières

Introduction **3**

I Mise en réseau et simulation du retard **4**

I.1 Architecture de communication 4

I.2 Format des messages et alignement mémoire 4

I.3 Émulateur de retard et mesure du T_{rc} 5

I.3.1 Analyse du cycle de vie via Wireshark 5

I.3.2 Analyse du retard effectif 6

II Identification des moteurs et analyse de stabilité **7**

II.1 Modèle du premier ordre avec retard 7

II.2 Protocole d'identification par réponse indicielle 7

II.2.1 Excitation des articulations 7

II.2.2 Traitement des données sous MATLAB 8

II.2.3 Reconstruction du modèle 8

II.3 Analyse de stabilité et commande adaptative 10

II.3.1 Condition de stabilité analytique 10

II.3.2 Mesure dynamique du retard via UDP 12

II.3.3 Dérivation de la loi de commande adaptative 12

II.4 Validation expérimentale 13

II.4.1 Situation stable (Correction adaptative active) 13

II.4.2 Situation mathématiquement instable mais convergente 14

II.4.3 Situation mathématiquement instable et divergente 15

III Implémentation logicielle et commande du bras **17**

III.1 Création d'un module de robotique de manipulation en C++ 17

III.2 Architecture de la bibliothèque `robManip` 17

III.2.1 Modélisation objet du robot 17

III.2.2 Modularité et abstraction des Entrées/Sorties 18

III.3 Commande Cinématique et Stabilité 19

III.3.1 Adaptation de la commande par retour d'état 19
III.3.2 Intégration de l'évitement de butées 19

Introduction

Ce projet porte sur le contrôle à distance d'un bras manipulateur six axes simulé dans CoppeliaSim. L'objectif est d'étudier l'impact d'un retard réseau sur la stabilité du système de commande, d'identifier les caractéristiques dynamiques des moteurs articulaires, et de mettre en œuvre les lois de commande nécessaires au déplacement du bras.

L'architecture logicielle repose sur une communication UDP entre trois entités distinctes : un client de commande, un émulateur de retard réseau, et un serveur interfacé avec le simulateur. Ce découplage permet d'injecter un retard configurable de manière transparente, sans modifier ni la logique de commande ni celle de simulation.

Le rapport est organisé en trois grandes parties :

1. la mise en réseau des processus et la simulation du retard de transmission ;
2. l'identification des moteurs et l'analyse de la stabilité en présence de retard ;
3. la modélisation du robot et les stratégies de commande.

Chapitre I

Mise en réseau et simulation du retard

I.1 Architecture de communication

Le système est structuré autour de trois processus communiquant par sockets UDP, simulant une boucle de contrôle distante :

- **Le client** (`client_robot.cpp`) : Calcule les consignes et mesure le retard.
- **Le bloc retard** (`retard_robot.c`) : Intercepte les paquets sur le port 2002 et les retransmet au serveur après un délai ajustable.
- **Le serveur de simulation** (`serveur_robot.cpp`) : Interface avec CoppeliaSim, traite la commande et renvoie l'état du robot au client sur le port 2003.

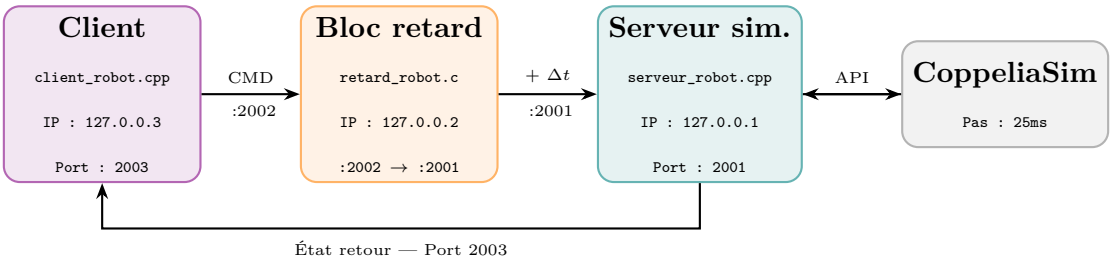

FIGURE I.1 – Architecture réseau et flux de données UDP.

I.2 Format des messages et alignement mémoire

La structure `msg_t` est partagée par tous les modules. Bien que la somme théorique des types soit de 164 octets ($4 + 3 \times 48 + 16$), la taille réelle observée dans les trames réseau est de 168 octets.

```c
typedef struct {
    int     cmdType;            // 4 octets (+ 4 octets de padding)
    double  cmd[6];             // 48 octets
    double  q_simu[6];          // 48 octets
    double  qdot_simu[6];       // 48 octets
    struct timeval time;        // 16 octets (Timestamp)
} msg_t;
```

Listing I.1 – Structure de données et contraintes d'alignement.

Ce décalage de 4 octets est dû au *padding* inséré par le compilateur après le champ `cmdType` pour aligner les champs `double` sur des adresses multiples de 8 octets.

I.3 Émulateur de retard et mesure du $T_{\mathbf{rc}}$

Le bloc de retard utilise un tampon circulaire pour stocker les messages reçus et calcule leur date de sortie. Pour obtenir un contrôle adaptatif, le client effectue une mesure dynamique T_{rc} en comparant l'horodatage `time` renvoyé par le serveur avec l'heure actuelle.

I.3.1 Analyse du cycle de vie via Wireshark

Pour isoler le trajet d'un message spécifique à travers les trois ports (`2002` → `2001` → `2003`), nous utilisons un filtre basé sur l'horodatage. Comme le serveur modifie les données de position mais pas le timestamp, ce dernier sert de "clé primaire". Le filtre Wireshark suivant permet d'isoler les 16 derniers octets de la charge utile UDP (offset 152) :

```
udp.payload[152:16] == [Valeur_Hexa_du_Timestamp]
```

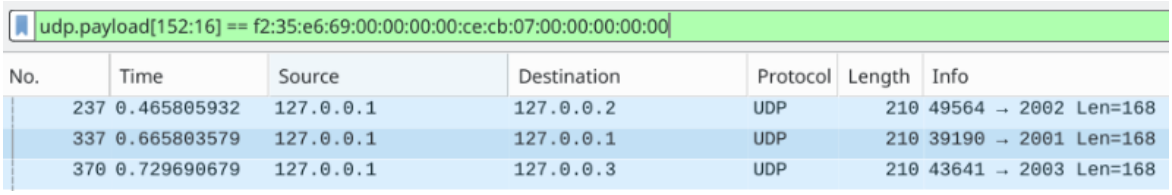

FIGURE I.2 – Capture Wireshark montrant le cycle de vie d'une commande. On identifie l'envoi client, la sortie du retardateur et le retour serveur partageant le même horodatage.

I.3.2 Analyse du retard effectif

On observe expérimentalement que le retard réel mesuré par le client est supérieur d'environ 50 ms au retard configuré dans l'émulateur. Cet écart s'explique par la somme de deux facteurs internes :

1. **Cycle de simulation :** Le serveur attend que CoppeliaSim effectue son pas de calcul ($dt = 25\,\text{ms}$) avant de pouvoir lire les nouvelles positions et renvoyer le paquet.
2. **Délais de lecture :** Les délais de lecture et d'envoi de chacun des blocs, introduisent une latence résiduelle d'environ 15 à 30 ms.

Le client utilise cette mesure globale (incluant les latences réseau et de simulation) pour mettre à jour le gain K_c avant chaque calcul de commande, garantissant ainsi la stabilité face aux fluctuations imprévues.

Chapitre II

Identification des moteurs et analyse de stabilité

II.1 Modèle du premier ordre avec retard

Chaque articulation est modélisée comme un système du premier ordre avec retard pur en boucle ouverte. La fonction de transfert en vitesse est :

$$G(s) = \frac{K\, e^{-T_{\mathrm{rc}} s}}{\tau s + 1} \tag{II.1}$$

où K est le gain statique, τ la constante de temps et T_{rc} le retard constant. Ce retard regroupe l'ensemble des délais de la chaîne d'acquisition, de transmission réseau et d'actionnement.

II.2 Protocole d'identification par réponse indicielle

II.2.1 Excitation des articulations

Le programme `step_ident.cpp` réalise une identification articulation par articulation. Pour chaque axe, la procédure est la suivante :

1. Le robot est ramené en position de référence (angles nuls).
2. Une phase de repos ($T_{\mathrm{rest}} = 5\,\mathrm{ms}$) à vitesse nulle permet d'établir les conditions initiales.
3. Un échelon de vitesse d'amplitude V_{step}, variable selon l'articulation, est appliqué pendant $T_{\mathrm{excite}} = 40\,\mathrm{ms}$.
4. Les données (temps, commande, vitesse mesurée) sont enregistrées dans un fichier CSV avec une période d'échantillonnage $dt = 1\,\mathrm{ms}$.

Le temps utilisé pour l'identification est le temps de simulation de CoppeliaSim, obtenu via `simxGetLastCmdTime`, ce qui garantit une cohérence avec le pas de simulation quelle que soit la charge du processeur hôte. Ainsi, on peut se permettre d'avoir sur CoppeliaSim un *real-time factor* inférieur à 1 sans que cela fausse nos mesures. Au final, un pas de simulation plus fin nous garantit une bien meilleure précision pour caractériser les articulations.

II.2.2 Traitement des données sous MATLAB

Le script `identification_1erOrdre.m` réalise l'identification en trois étapes :

Détection du début de l'échelon et de la réaction de la sortie : Le début de l'échelon est détecté lorsque l'entrée dépasse 5 % de son amplitude finale. La réaction de la sortie à $t = t_{\text{début_modèle}}$, est détectée quand celle-ci s'écarte de plus de 2 % de sa valeur initiale.

Pour l'identification des moteurs, même sans bloc de retard, on observe un décalage de la sortie par rapport à l'entrée, proche des 2 ms. Cela est du au délai de communication avec le simulateur CoppeliaSim auquel on additionne un pas de simulation ($dt = 1\,\text{ms}$).

Identification du gain statique K : Le gain est calculé à partir des valeurs finales :

$$K = \frac{y_\infty - y_0}{u_\infty} \tag{II.2}$$

Identification de la constante de temps τ : La méthode des 63 % est appliquée sur la fenêtre temporelle décalée du retard. L'instant auquel la sortie atteint $y_0 + 0{,}632 \cdot (y_\infty - y_0)$ donne directement τ.

II.2.3 Reconstruction du modèle

Le modèle reconstruit est :

$$y_{\text{modèle}}(t) = \begin{cases} y_0 & \text{si } t < t_{\text{début_modèle}} \\ y_0 + K\,u_\infty \left(1 - e^{-(t - t_{\text{début_modèle}})/\tau}\right) & \text{sinon} \end{cases} \tag{II.3}$$

Les paramètres identifiés pour les six articulations sont résumés dans le tableau II.1. Afin d'effectuer les mesures, on a prit un échelon de $0.05 rad/s$ ou bien $0.1 rad/s$ selon les articulations, afin d'éviter qu'elles ne saturent (accélération maximale). En cas de saturation, on observe une droite et non une exponentielle donc le modèle ne serait pas valide.

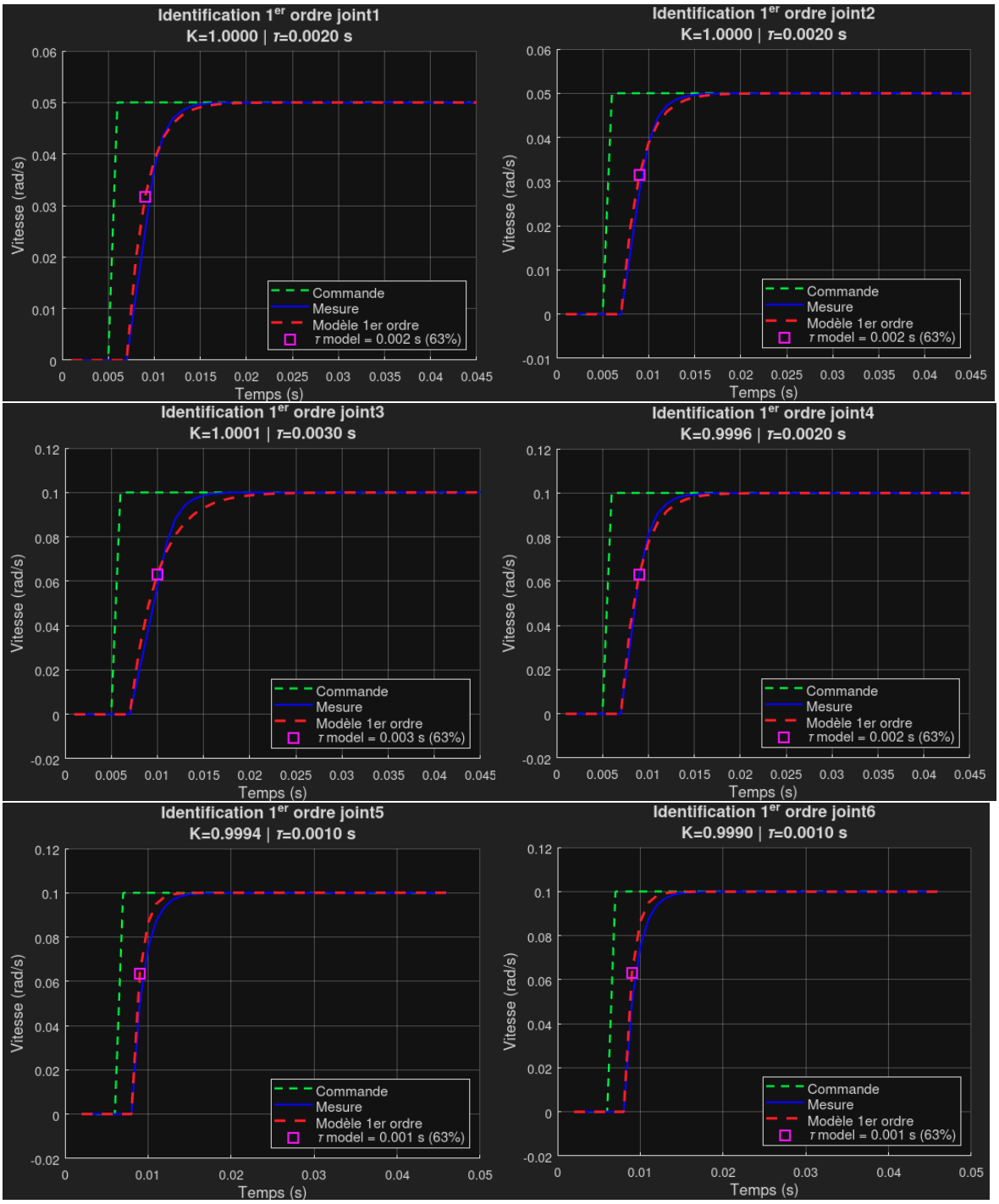

FIGURE II.1 – Identification et modélisation des 6 joints

Table II.1 – Paramètres identifiés par articulation.

Articulation	K (ou G)	$\tau_{63\%}$ (s)
Joint 1	1.00	0,002
Joint 2	1.00	0,002
Joint 3	1.00	0,003
Joint 4	1.00	0,002
Joint 5	1.00	0,001
Joint 6	1.00	0,001

II.3 Analyse de stabilité et commande adaptative

II.3.1 Condition de stabilité analytique

Pour un système du premier ordre avec retard en boucle fermée, la limite de stabilité est définie par le couple gain/retard pour lequel le système entre en oscillation entretenue (marge de phase nulle). Analytiquement, on établit la relation liant le retard maximal admissible $T_{\text{rc}}^{\max}$ au produit K_cG (où G est le gain statique) :

$$T_{\text{rc}}^{\max} = \frac{\tau \arccos\left(-\dfrac{1}{K_cG}\right)}{\sqrt{(K_cG)^2 - 1}} \tag{II.4}$$

Cette relation n'est définie que pour $K_cG > 1$. En dessous de cette valeur, le système est inconditionnellement stable. La figure II.2 présente les limites de stabilité pour différentes constantes de temps τ. La région de stabilité se situe sous chaque courbe.

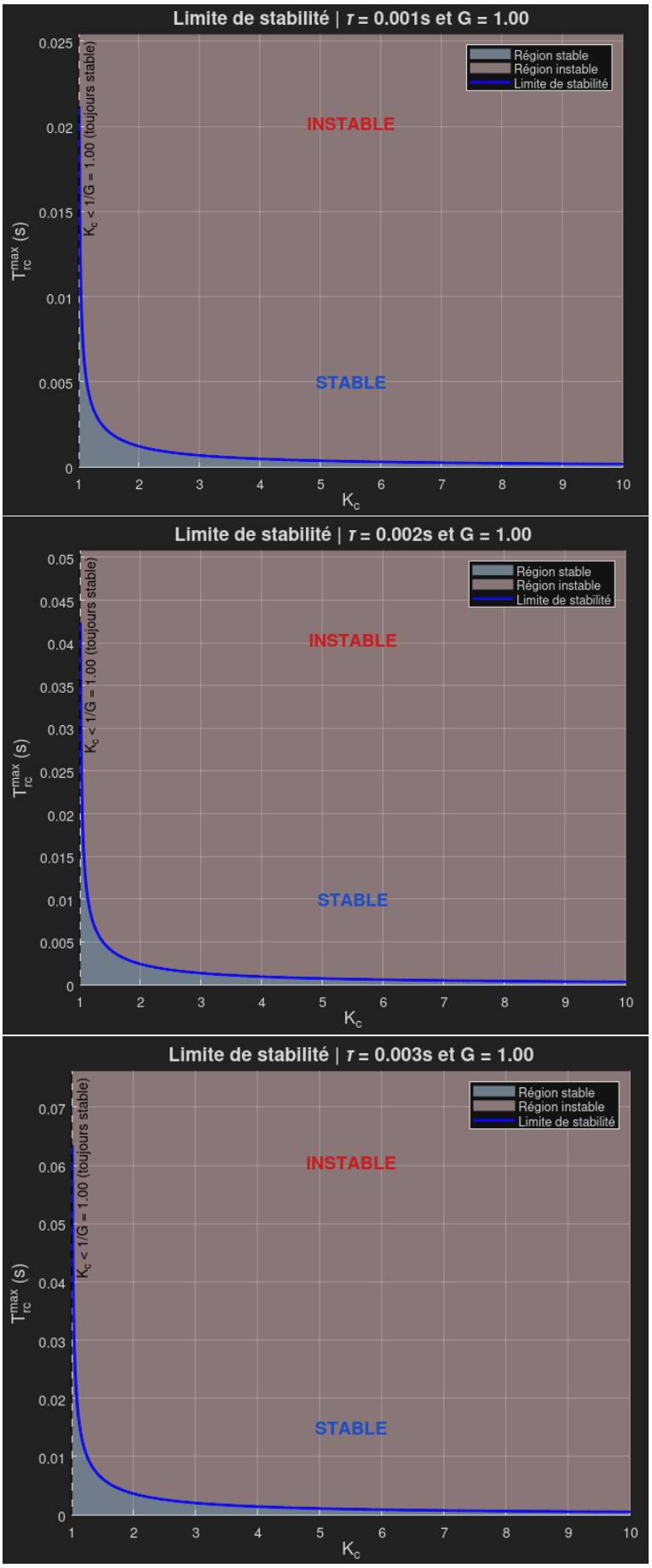

FIGURE II.2 – Limites de stabilité pour $\tau = \{1, 2, 3\}\, ms$ ($G = 1.00$).

II.3.2 Mesure dynamique du retard via UDP

Pour que la loi de commande puisse s'adapter, le client doit connaître en temps réel la valeur du retard T_{rc}. Ce calcul repose sur un mécanisme d'écho : le client horodate chaque paquet (`msg.time`) et le serveur lui renvoie cet horodatage sans modification.

Comme analysé précédemment via les trames Wireshark (voir Figure I.2), ce retard englobe le transit réseau (aller-retour) ainsi que le pas de simulation de CoppeliaSim ($25\,ms$).

```
// Get current time
gettimeofday(&now, NULL);

// Read current positions (also stores the latest message from
    the server in latest_message_serveur)
if (getAllJointsPosition(0, 0, &q_mes) != ERROR) {
    // Record the current message
    logger.record(&latest_message_serveur);

    // Measure dynamic Trc in seconds
    double Trc = diffTime_ms(&latest_message_serveur.time, &now
        ) / 1000.0;

    // To prevent division by zero
    if (Trc <= 0.001) Trc = 0.001;
```

Listing II.1 – Calcul du T_{rc} effectif dans la boucle de contrôle

II.3.3 Dérivation de la loi de commande adaptative

Dans notre architecture, nous asservissons le robot en position via des consignes de vitesse. La fonction de transfert en boucle ouverte $H_{BO}(s)$ intègre donc un intégrateur pur, la dynamique moteur et le retard mesuré :

$$H_{BO}(s) = \frac{K_c \cdot G}{s(1+\tau s)} e^{-T_{rc}s} \tag{II.5}$$

La stabilité critique est atteinte à la pulsation ω_u lorsque la phase cumulée du système atteint $-\pi$. Chaque composant contribue au retard de phase comme suit :

$$\underbrace{-\frac{\pi}{2}}_{\text{Intégrateur}} \underbrace{-\arctan(\tau\omega_u)}_{\text{Moteur}} \underbrace{-T_{rc}\omega_u}_{\text{Retard UDP}} = -\pi \quad \Rightarrow \quad \arctan(\tau\omega_u) + T_{rc}\omega_u = \frac{\pi}{2} \tag{II.6}$$

Compte tenu de la faible valeur de τ (1 à $3\,ms$), on peut utiliser l'approximation des petits angles ($\arctan(x) \approx x$) pour isoler la pulsation critique :

$$\omega_u \cdot (\tau + T_{rc}) \approx \frac{\pi}{2} \quad \Rightarrow \quad \omega_u \approx \frac{\pi}{2(T_{rc} + \tau)} \tag{II.7}$$

À cette pulsation ω_u, le module de $H_{BO}(j\omega_u)$ doit être unitaire pour définir le gain maximal $K_{\max}$. En extrayant $K_{\max}$ de l'équation du module, on obtient :

$$\frac{K_{\max} \cdot G}{\omega_u \sqrt{1 + (\tau\omega_u)^2}} = 1 \quad \Rightarrow \quad K_{\max}(T_{rc}) = \frac{\omega_u}{G}\sqrt{1 + (\tau\omega_u)^2} \tag{II.8}$$

Enfin, pour garantir une marge de phase suffisante et éviter les dépassements oscillatoires, le gain appliqué K_c est pondéré par un coefficient de sécurité $\alpha < 1$:

$$K_c(T_{rc}) = \alpha \cdot K_{\max}(T_{rc}) \tag{II.9}$$

Cette loi permet de recalculer la raideur de l'asservissement avant chaque envoi de consigne, assurant ainsi que le robot reste stable même en cas de changement du retard introduit par le réseau.

II.4 Validation expérimentale

Afin de valider notre loi de commande, nous avons analysé le comportement du robot face à des retards constants dans trois configurations distinctes de gain.

II.4.1 Situation stable (Correction adaptative active)

Lorsque la loi adaptative est activée avec un coefficient de sécurité $\alpha = 0,4$, le système compense automatiquement l'augmentation du retard réseau en diminuant sa raideur (baisse du gain K_c).

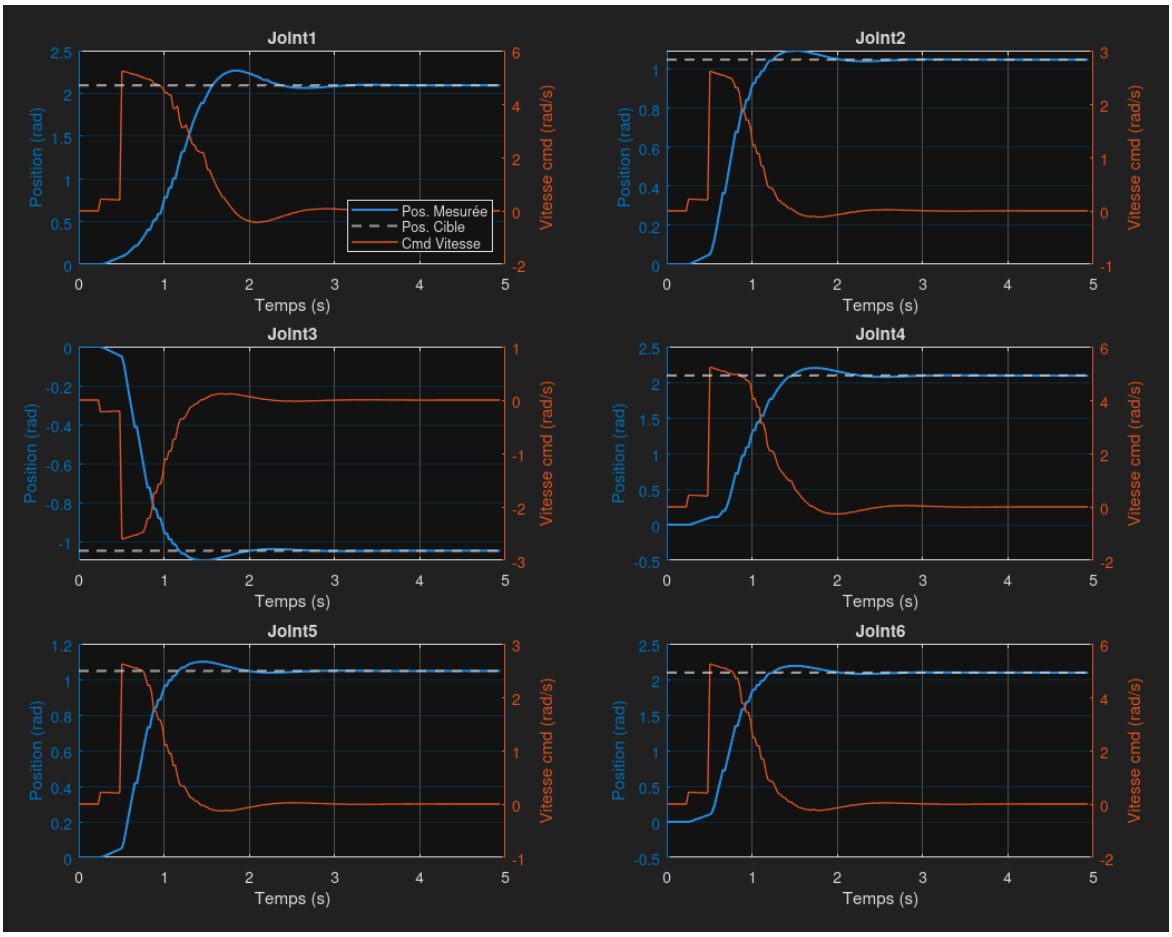

FIGURE II.3 – Réponse indicielle stable des articulations vers la position cible avec un retard réseau de 200 ms.

Comme l'illustre la Figure II.3, le robot atteint ses consignes articulaires finales (q_f) avec une convergence fluide. On observe un léger dépassement mais une absence totale d'oscillations entretenues.

II.4.2 Situation mathématiquement instable mais convergente

Pour mettre en évidence les limites du système, nous avons forcé le paramètre α à $1,1$. Ce réglage impose un gain de boucle $K_cG > 1$, plaçant théoriquement le robot dans une zone d'instabilité.

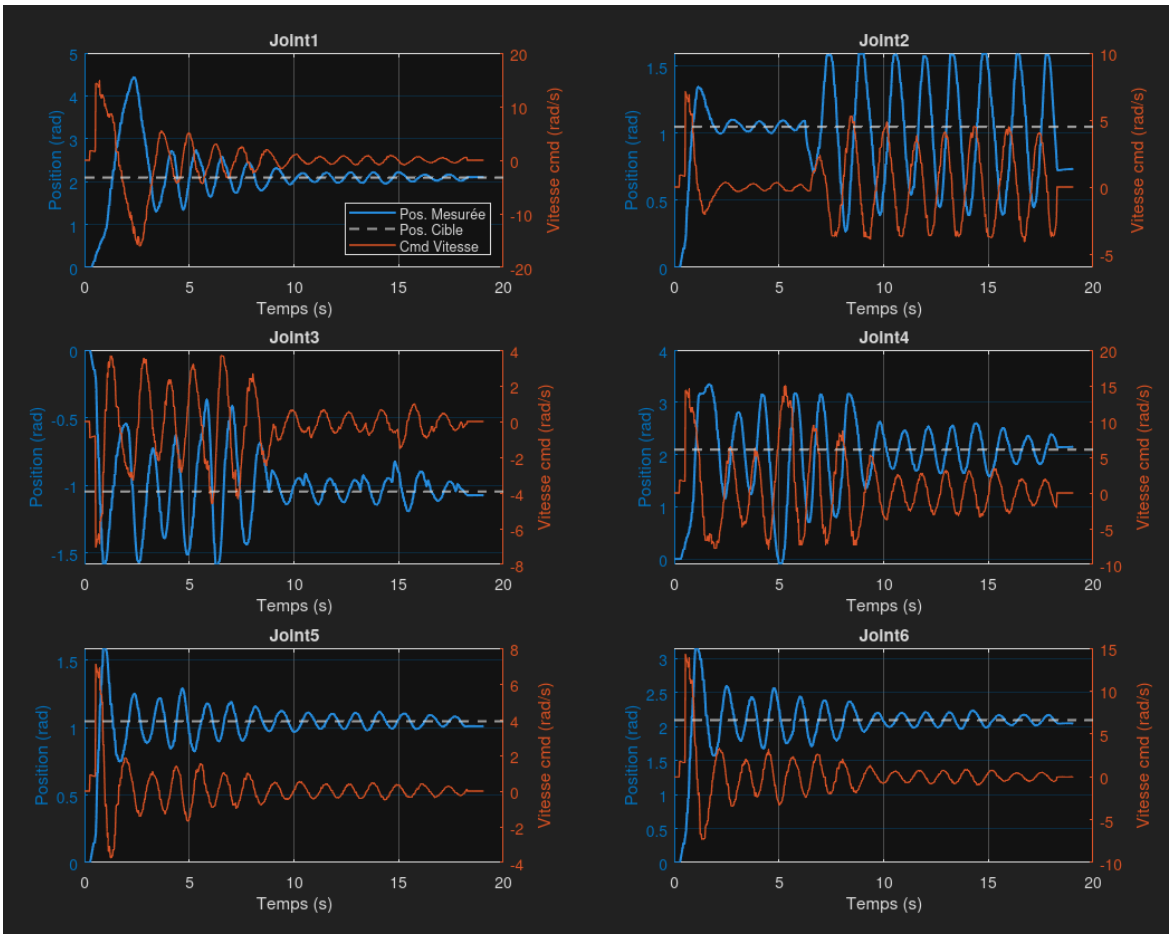

FIGURE II.4 – Oscillations observées pour $\alpha = 1,1$. Les non-linéarités limitent l'amplitude des oscillations.

L'analyse de la Figure II.4 révèle un comportement particulier : contrairement au modèle linéaire pur qui prédirait une divergence infinie, les oscillations s'atténuent progressivement jusqu'à la convergence. Ce phénomène s'explique par les non-linéarités implémentées dans le module `robManip.cpp` (limites de vitesse $V_{\max}$ et d'accélération $a_{\max}$) qui agissent ici comme des dissipateurs d'énergie.

Il est à noter que l'arrêt de la commande après 15 s provient du *timeout* de sécurité du `client_robot`. Si la cible n'est pas atteinte dans ce délai, la vitesse est forcée à 0 rad/s tandis que le client continue d'écouter le serveur pendant 1 s pour stabiliser l'état final.

II.4.3 Situation mathématiquement instable et divergente

En poussant le gain à une valeur extrême ($\alpha = 1,5$), le système franchit le seuil de divergence réelle. Ici, l'énergie injectée par le correcteur à chaque cycle surpasse les capacités de dissipation des limites de vitesse.

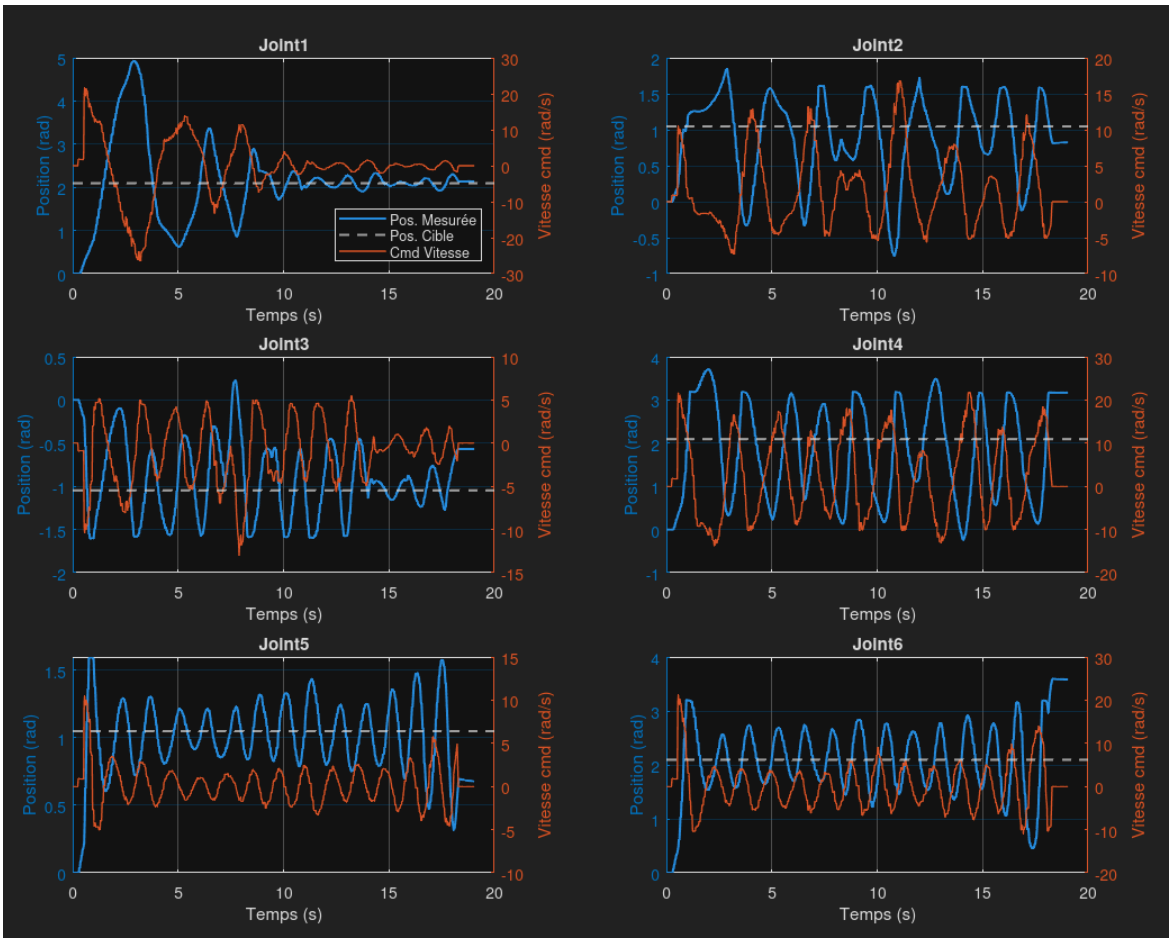

FIGURE II.5 – Instabilité et divergence des oscillations pour $\alpha = 1,5$.

L'analyse des signaux montre une opposition de phase quasi totale entre la commande et la position mesurée. À cause du retard T_{rc}, le correcteur applique une vitesse maximale pour compenser une erreur perçue alors que le bras a déjà dépassé sa cible. Cette inversion transforme l'asservissement en un injecteur d'énergie qui amplifie l'oscillation à chaque période.

Sur la Figure II.5, l'amplitude des oscillations augmente pour plusieurs articulations mais elle reste limitée par les butées articulaires $(\theta_{\min}, \theta_{\max})$. Les limites articulaires de chacun des joints sont les suivantes :

$$\theta_{\min} = \{-\pi, -\frac{\pi}{2}, -\frac{\pi}{2}, -\pi, -\frac{\pi}{2}, -\pi\} \quad \text{et} \quad \theta_{\max} = \{\pi, \frac{\pi}{2}, \frac{3\pi}{4}, \pi, \frac{\pi}{2}, \pi\}$$

Le système ne peut alors plus diverger mathématiquement, mais il percute violemment ses limites mécaniques. Cela confirme la nécessité de maintenir le produit K_cG sous un seuil critique pour éviter que le déphasage global n'atteigne les $-180°$.

Le dépassement affiché des limites mécaniques sur certaines figures est probablement dû à une erreur de mesure du simulateur ou à une mauvaise configuration.

Chapitre III

Implémentation logicielle et commande du bras

III.1 Création d'un module de robotique de manipulation en C++

Ce projet s'inscrit dans une démarche de portage en C++ d'un environnement de simulation initialement développé sous Julia. L'objectif n'est plus ici de démontrer les fondements théoriques de la robotique (déjà validés dans des projets antérieurs [1] et [2]), mais de proposer une implémentation modulaire et simple à utiliser en C++.

Un des avantages majeurs du C++ par rapport à Julia ou MATLAB est la possibilité de représenter un robot sous forme d'objet. On va donc tirer profit de cette fonctionnalité.

Pour faciliter les calculs matriciels, on utilisera la bibliothèque `Eigen`.

III.2 Architecture de la bibliothèque `robManip`

III.2.1 Modélisation objet du robot

Le cœur du module repose sur la classe `Robot`. Contrairement à une approche procédurale, cette classe agrège l'ensemble des constantes physiques et des variables d'état d'un bras robot. De plus, elle met à disposition plusieurs méthodes pour réaliser différents types de commandes (trapèze et cinématique).

```
class Robot {
private:
    Eigen::VectorXd theta, alpha, d, r; // Parametres DH
    Eigen::VectorXd theta     // Offset sur chaque articulation
    Eigen::VectorXd q_min, q_max;  // Limites articulaires
    Eigen::VectorXd V_max          // Limites de vitesse
    Eigen::VectorXd a_max          // Limites d'acceleration
    Eigen::Matrix4d T_tool;        // Transformation de l'outil
    [...]

public:
    std::pair<Matrix4d, vector<Matrix4d>> MGD() const;
    Eigen::MatrixXd Jacobienne(const Eigen::Vector3d& P);
    void simuTrapeze(...);
    void cmdCinematique(...);
    [...]
};
```

Listing III.1 – Extrait de la classe Robot dans `robManip.hpp`

Cette structure permet d'instancier plusieurs robots au sein d'une même simulation. Notamment avec l'ajout de fonctions qui initialisent un robot donné avec ses paramètres DH remplis automatiquement. Par exemple, `CreateRobotisH(const Eigen::VectorXd& initialTheta)`, rempli les paramètres DH du Robotis-H 6ddl, avec un crayon positionné sur son effecteur. Il s'agit du modèle utilisé durant ce projet.

III.2.2 Modularité et abstraction des Entrées/Sorties

Un choix de conception a été fait concernant les fonctions de bas niveau `sendCmd()` et `getAllJointsPosition()`. Bien qu'essentielles, ces fonctions ne sont pas implémentées au sein de la bibliothèque `robManip`.

Justification du choix : La bibliothèque est conçue pour être indépendante du matériel ou du simulateur utilisé. En laissant l'implémentation de ces fonctions à la charge de l'utilisateur (ici, dans le `client_robot.cpp`), le module `robManip` reste indépendant des API spécifiques (comme les RemoteAPI de CoppeliaSim). Cela facilite le portage du code d'un environnement simulé vers un robot réel sans modifier le moteur de calcul cinématique.

III.3 Commande Cinématique et Stabilité

III.3.1 Adaptation de la commande par retour d'état

L'implémentation de la commande cinématique en C++ reprend la structure en boucle fermée définie dans un projet antérieur sur le `Robotis-H 6ddl` [1].

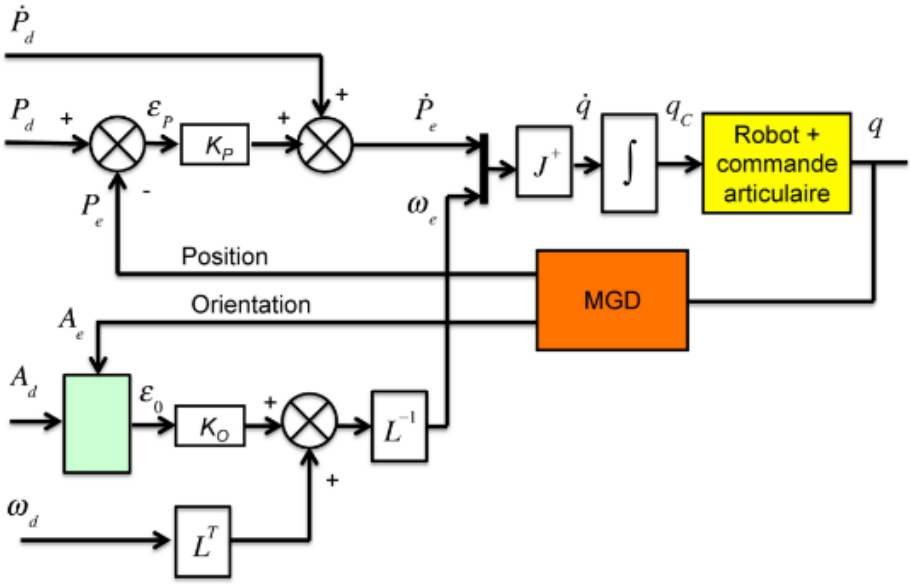

FIGURE III.1 – Schéma bloc de la commande cinématique

III.3.2 Intégration de l'évitement de butées

Nous avons implémenté une fonction utilisant le noyau de la Jacobienne pour l'évitement des butées articulaires. Cette fonction provient d'un projet antérieur sur le robot `KUKA LWR` [2].

$$\dot{\theta} = J^+\dot{X} + (I - J^+J)\alpha\nabla\phi$$

$$\nabla\phi_i = \frac{2(\theta_\mathrm{i} - \theta_\mathrm{i,moy})}{\theta^2_\mathrm{i,range}}$$

$$\phi_i(\theta) = \frac{(\theta_\mathrm{i} - \theta_\mathrm{i,moy})^2}{\theta^2_\mathrm{i,range}}$$

$$\theta_\mathrm{i,moy} = \frac{\theta_\mathrm{i,max} + \theta_\mathrm{i,min}}{2} \quad \text{et} \quad \theta_\mathrm{i,range} = \theta_\mathrm{i,max} - \theta_\mathrm{i,min}$$

Bibliographie

[1] MUSUMECI Lucas, *Projet Robotique - Robotis-H 6ddl*, Rapport de projet, Polytech Montpellier, 30 novembre 2025. https://github.com/lucasmusumeci/6DoF_Robotis-H_Manipulator/blob/main/Rapport_ProjetRobS7-1.pdf

[2] MUSUMECI Lucas, *Mini-Projet robot KUKA LWR*, Rapport de projet, Polytech Montpellier, 8 Janvier 2026. https://github.com/lucasmusumeci/7Dof_KUKA_LWR/blob/master/Rapport_KUKA_LWR.pdf